

1. (10%) This exercise examines how pipelining affects the clock cycle time of the processor. Assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
400ps	450ps	450ps	600ps	550ps

- (1) (3%) What is the clock cycle time in a pipelined and non-pipelined processor?

Pipeline : 600 ps

Non-Pipeline : $400 + 450 + 450 + 600 + 550 = 2450$ ps

- (2) (3%) What is the total latency of an LW instruction in a pipelined and non-pipelined processor?

Pipeline : $600 \times 5 = 3000$ ps

Non-Pipeline : $400 + 450 + 450 + 600 + 550 = 2450$ ps

- (3) (4%) If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

Split MEM , New Cycle : 550 ps.

3. (15%) Consider instruction executing in the processor implemented by Figure 6. following with:

PC	instruction
8004	loop: lw r1,8(r1)
8008	add r2,r2,r3
8012	sub r3,r4,r5
8016	beq r4,r0,loop

Please fill in the **decimal value** of (a),(b),(c),(d),(e) in the first seven cycles.**If instruction in the current stage is not used, fill in X.**

Assume the **address of the first instruction is 8004**, and that **all values in Memory are 0**, and values of register are following with Table 4.

(Hint: For the output of each MUX, you should determine it through the instruction in the current Stage.)

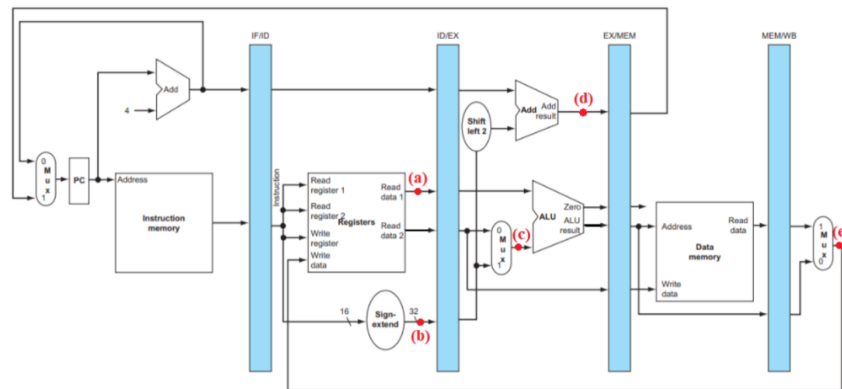


FIGURE 4.35 The pipelined version of the datapath in Figure 4.33. The pipeline registers, in color, separate each pipeline stage. They are labeled by the stages that they separate; for example, the first is labeled IF/ID because it separates the instruction fetch and instruction decode stages. The registers must be wide enough to store all the data corresponding to the lines that go through them. For example, the IF/ID register must be 64 bits wide, because it must hold both the 32-bit instruction fetched from memory and the incremented 32-bit PC address. We will expand these registers over the course of this chapter, but for now the other three pipeline registers contain 128, 97, and 64 bits, respectively.

Figure 6.

r0	r1	r2	r3	r4	r5
8	48	1	3	2	5

Table 4.

lw	IF	ID	EX	MEM	WB			
add		IF	ID	EX	MEM	WB		
sub			IF	ID	EX	MEM	WB	
beq				IF	ID	EX	MEM	WB

	(a)	(b)	(c)	(d)	(e)
Cycle 1	X	X	X	X	X
Cycle 2	48	8	X	X	X
Cycle 3	1	X	8	X	X
Cycle 4	2	X	3	X	X
Cycle 5	2	-4	5	X	0
Cycle 6	X	X	8	8004	4
Cycle 7	X	X	X	X	-3

4. (10%) Consider instruction executing in the processor implemented by Figure 7 and following the below instruction sequence.

- (1) (4%) Please refer to Table 5. and complete the tasks for **cycle 3** by filling the instructions in (a), (b), and (c), and filling the decimal values for the control signals in (d), (e), (f), (g), and (h).

Hint: Control signals (d), (e), (g) are composed by:

(d)=[RegWrite, MemtoReg] ,

(e)=[Branch, MemRead, MemWrite],

(g)=[ALUOp1, ALUOp0]

Instruction sequence	
1	sub \$11, \$2, \$3
2	lw \$10, 20(\$1)
3	and \$11, \$4, \$11
4	or \$13, \$11, \$7
5	add \$14, \$8, \$9

(d) 10
(e) 000
(f) 1
(g) 10
(h) 0

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Table 5

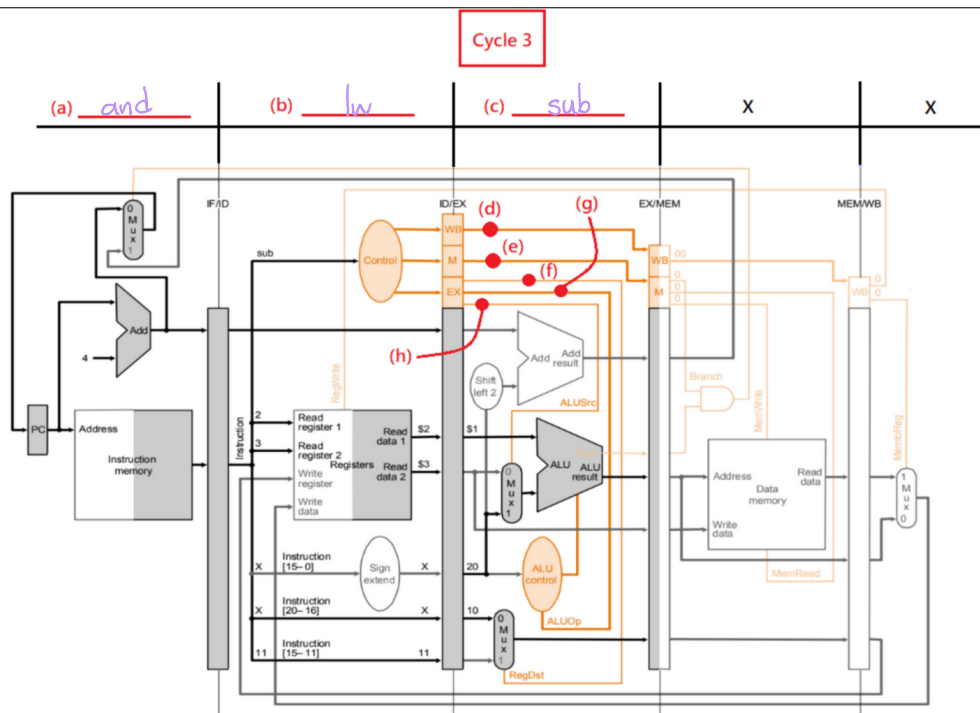


Figure 7.

(2) (6%) If there is forwarding, for the **first eight cycles** during the execution of this code, specify ForwardingA and ForwardingB shown in Figure 8. of each cycle by **forwarding units** in Figure 9.

		1	2	3	4	5	6	7	8	
sub		IF	ID	EX	MEM	WB				
lw			IF	ID	EX	MEM	WB			
and				IF	ID	EX	MEM	WB		
or					IF	ID	EX	MEM	WB	
add						IF	ID	EX	MEM	WB
Forwarding A	A	X	X	00	00	00	10	00	X	
Forwarding B	B	X	X	00	X	01	00	00	X	
						↳ lw	NO	rt		

5.(10%) About the MIPS with Static Dual Issue, which of the following statements are True/False? And give the reasons for each answer.

- (a) (2%)A packet contains two sets of instructions: one dedicated to load/store operations and the other to ALU/branch operations.
- (b) (2%)A packet can be implemented by adding just one ALU.
- (c) (2%)Two-issue packets require additional register ports.
- (d) (2%)Two-issue packets, within one cycle, need to read two registers for ALU operations and two registers for storage operations. One write port is used for ALU operations, and another write port is used for load operations.
- (e) (2%)Use latency refers to the number of clock cycles between a load instruction and the instruction that can use the result of the load with stalling the pipeline.

Ans: True : (a)(c)(d)

False : (b) 還需要一個 sign extender (e)是 without stalling the pipeline.

6. (10%) Consider the below code sequence. Please use the technique of **loop unrolling** to duplicate this code sequence **three times** and optimally schedule the unrolled code sequence to a **static dual issue** pipelined MIPS CPU where one packet is for ALU/branch instructions and the other is for lw/sw instructions. **Show the answer of ALU/branch instructions and lw/sw instructions of each cycle.**

instruction
Loop: lw \$t0, 0(\$s1) addu \$t0, \$t0, \$s2 sw \$t0, 0(\$s1) addi \$s1, \$s1, -4 bne \$s1, \$zero, Loop

LABEL	ALU/branch	Load/Store	cycle
Loop:	addi \$s1, \$s1, -12	lw \$t0, 0(\$s1)	1
	nop	lw \$t1, 8(\$s1)	2
	addu \$t0, \$t0, \$s2	lw \$t2, 4(\$s1)	3
	addu \$t1, \$t1, \$s2	sw \$t0, 12(\$s1)	4
	addu \$t2, \$t2, \$s2	sw \$t1, 8(\$s1)	5
	bne \$s1, \$zero, Loop	sw \$t2, 4(\$s1)	6

7. (10%) This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, T, NT, NT, T, NT, NT

7.1.(5%) What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?

ANS: (1) Taken : $\frac{3}{7} = 42.85\%$
 (2) Not-Taken : $\frac{4}{7} = 57.14\%$

7.2. (5%) What is the accuracy of the two-bit predictor for the first 4 branches in this pattern, assume that the predictor starts off in the bottom left state from Figure 4.63 (predict not taken)?

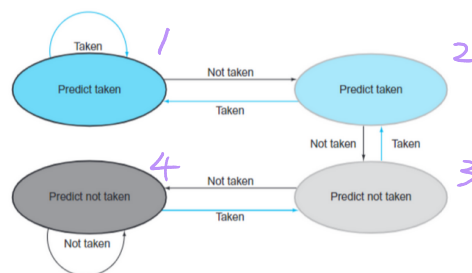


FIGURE 4.63 The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted only once. The 2 bits are used to encode the four states in the system. The 2-bit scheme is a general instance of a counter-based predictor, which is incremented when the prediction is accurate and decremented otherwise, and uses the mid-point of its range as the division between taken and not taken.

Steps	1	2	3	4
Guess	NT	NT	T	NT
Real	T	T	NT	NT
Correct	×	×	×	✓
State (4)	3	2	3	4

A: 25%

8.(15%) To explore how exception handling affects pipeline design. The first three problems refer to the following two instructions:

Instruction 1	Instruction 2
beq r1, r2, L1	sw r1, 0(r1)

- (1) (3%) Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected.

A: (1) beq : Invalid target address (EX)

(2) sw : Misaligned memory address (MEM)

- (2) (3%) If there is a separate handler address for each exception, show how the pipeline organization must be changed to be able to handle this exception. You can assume that the addresses of these handlers are known when the processor is designed.

A: 在 PC address 輸入端加入一個 MUX 選擇不同 handler 的進入位址，在發生錯誤時依情況選擇對應的 handler.

- (3) (3%) If the second instruction is fetched right after the first instruction, describe what happens in the pipeline when the first instruction causes the first exception you listed in (1). Show the pipeline execution diagram from the time the first instruction is fetched until the time the first

A: IF、ID、EX 階段執行 Flush 清除錯誤資料並避免後續指令執行，WB 階段則正常寫回，PC 被設定為進入 handler.

- (4) (3%) In vectored exception handling, the table of exception handler addresses is in data memory at a known (fixed) address. Change the pipeline to implement this exception handling mechanism. Repeat (3) using this modified pipeline and vectored exception handling.

A: (1) 從 MEM 中讀取 exception handler table.

(2) 在 table 中搜尋對應 exception handler 的位址並 jump

(3) 在 EX 階段計算 exception handler table 的位址，

MEM 讀取 exception handler 的進入位址，

WB 寫回 PC handler 位址

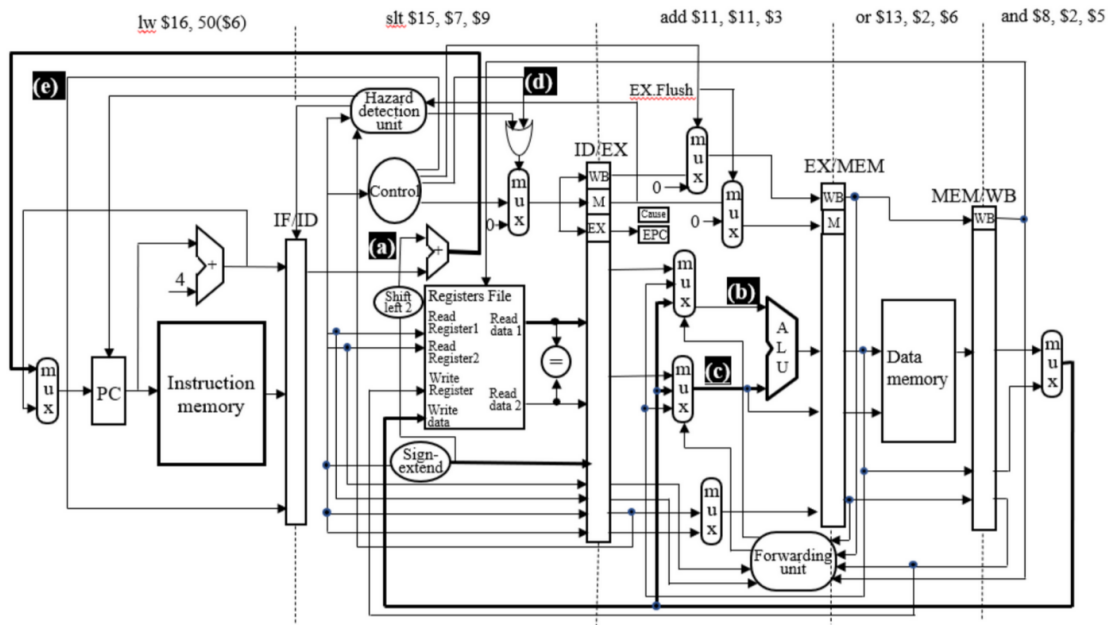
(5) (3%) We want to emulate vectored exception handling (described in (4)) on a machine that has only one fixed handler address. Write the code that should be at that fixed address. Hint: this code should identify the exception, get the right address from the exception vector table, and transfer execution to that handler.

```
sw    $s0, 0($sp)
add   $s0, $0, $EXCEPTION_VECTOR_TABLE
add   $s0, $s0, $CAUSE
j     $s0
```

9. (15%) For the 5 stages pipeline processor as the below figure, assume the following sequence of instructions:

Instructions	Notes
24 sub \$11, \$2, \$4	(a) is the 32bits binary value.
28 and \$8, \$2, \$5	(b), (c) are the register numbers.
2C or \$13, \$2, \$6	(d), (e) is the control signal.
30 add \$11, \$11, \$3	slt is R-type, funct = 42.
34 slt \$15, \$7, \$9	
38 lw \$16, 50(\$6)	

The "30 add" overflow is detected in EX. at the cycle 6. Please complete the following (a)~(e) in the pipelined datapaths and control.



(a) 00000000 00000001 11000000 10101000

(b) \$11

(c) \$3

(d) ID. Flush

(e) IF. Flush

	<u>op(6)</u>	<u>rs(5)</u>	<u>rt(5)</u>	<u>rd(5)</u>	<u>shamt(5)</u>	<u>funct(6)</u>	
slt :	0	5	9	15	0	42	
	<u>0111 0000 101010</u>						10

10. (15%) Consider a typical MIPS processor with a 5-stage pipeline datapath as follows.

- IF: the instruction fetch stage.
- ID: the instruction decode/register file read stage.
- EX: the execution stage.
- MEM: the memory access stage.
- WB: the write-back stage.

(1) (6%) The following MIPS code is executed on the above processor.

I	code
1	add \$s1, \$t1, \$t2
2	sub \$s1, \$s1, \$s2
3	lw \$s2, 0(\$s1)
4	add \$t1, \$s1, \$s2
5	sub \$t3, \$t2, \$s2
6	sw \$s1, 0(\$t3)
7	or \$t4, \$t2, \$s2
8	add \$t1, \$s2, \$s1

Assume forwarding and stall mechanisms have been designed. Please fill in the blanks in the following table when the code sequence runs to the 9 cycle.

	1	2	3	4	5	6	7	8	9			
I1	IF	ID	EX	MEM	WB							
I2		IF	ID	EX	MEM	WB						
I3			IF	ID	ID _(n)	EX	MEM	WB				
I4				IF	IF _(n)	ID	EX	MEM	WB			
I5					IF	ID	EX	MEM	WB			
I6						IF	ID	EX	MEM	WB		
I7							IF	ID	EX	MEM	WB	
I8								IF	ID	EX	MEM	WB

(2) (9%) The following MIPS code is executed on the above processor.

I	code
1	add \$s5, \$s3, \$s4
2	lw \$s2, 16(\$s1)
3	add \$s0, \$s2, \$s3
4	sub \$s0, \$s3, \$s4

(a) Find all data dependencies in the instruction sequence.

ANS: (1) RAW : (I2, I3 , \$s2)
(2) WAR : ✕
(3) WAW : (I3, I4 , \$s0)

(b) Find all hazards in the instruction sequence for the processor with and without forwarding.

ANS: (1) With Forwarding :
(I2, I3) : Load-use hazard.
(2) Without Forwarding :
(I2, I3) : Load-use 2 cycle stall

(c) Sometimes, even with forwarding, one stage would have to be stalled for a data hazard. The software technique, Reordering Code, would be adopted to avoid the pipeline stalls. Please state if the instruction sequence suffers from a data hazard that cannot be resolved by the forwarding, and list the reordered code.

ANS: (I2, I3) ... Load Use Hazard

lw \$s2, 16(\$s1)
add \$s5, \$s3, \$s4
add \$s0, \$s2, \$s3
sub \$s0, \$s3, \$s4